

Mi Ticketera (Kanban Board)

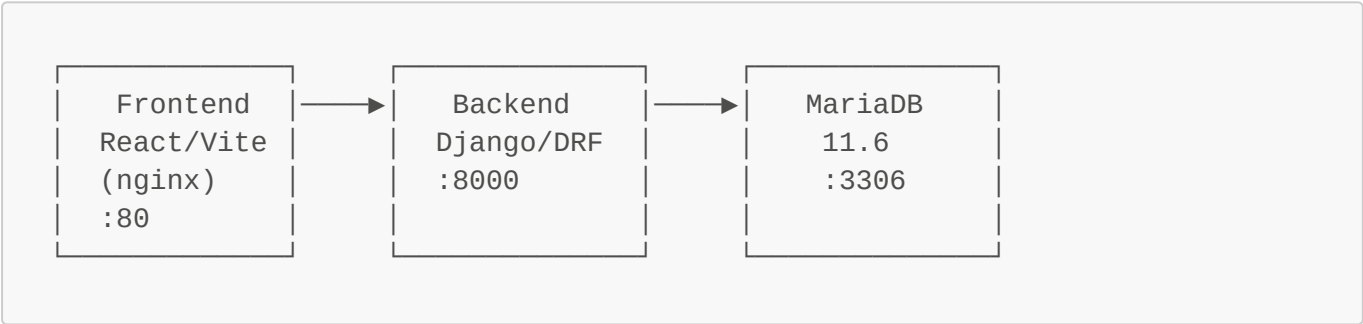
Sistema de gestión de tareas estilo Kanban para la Dirección de Informática — Delegación III (Provincia de Buenos Aires).

Objetivo

Reemplazar la dependencia de Supabase (BaaS) por una infraestructura local y autoadministrada:

- **Frontend:** React 19 + Vite 7 + Tailwind 4 (contenedorizado con nginx)
- **Backend:** Django 5.1 + Django REST Framework (Python 3.12)
- **Base de datos:** MariaDB 11.6
- **Orquestación:** Docker Compose (3 contenedores)

Arquitectura



Estructura del repositorio

```
ticketeraMCTI/
├── docker-compose.yml           # Orquesta los 3 contenedores
├── Dockerfile.frontend         # Build multi-stage: Node 20 → nginx:alpine
├── nginx.conf                  # SPA routing + cache headers
├── .env                        # Variables VITE_API_URL
├── .dockerignore
├──
├── src/                        # Código fuente React
│   ├── main.jsx
│   ├── App.jsx                 # Router principal
│   ├── api.js                  # Cliente HTTP contra Django REST API
│   ├── context/
│   │   └── AuthContext.jsx     # Autenticación (login/register/logout)
│   ├── pages/
│   │   ├── Login.jsx
│   │   ├── Registro.jsx
│   │   └── Dashboard.jsx       # CRUD tableros, perfil, notificaciones
│   ├── components/
│   │   ├── TableroKanban.jsx   # Tablero Kanban completo
│   │   ├── EstadisticasPanel.jsx
│   │   ├── GestionUsuariosGlobal.jsx
│   │   └── RutaProtegida.jsx
```

```
├── utils/
│   └── configTablero.js
├── backend/                                # Proyecto Django
│   ├── Dockerfile                        # Python 3.12-slim
│   ├── requirements.txt
│   ├── entrypoint.sh                    # migrate + collectstatic + createsuperuser
│   ├── ticketera/                       # Settings del proyecto Django
│   │   ├── settings.py
│   │   ├── urls.py
│   │   └── wsgi.py
│   ├── core/
│   │   ├── models.py                   # Modelos de dominio
│   │   └── admin.py
│   └── api/
│       ├── serializers.py
│       ├── views.py                    # ViewSets + auth endpoints
│       └── urls.py                     # Router REST
├── db/
│   └── init.sql                         # Inicialización de MariaDB
└── n8n/                                # Plantillas de email (legacy)
```

Modelos de datos

Ver [DER.md](#) para el diagrama entidad-relación completo.

Tabla	Descripción
usuarios	Perfiles de usuario (extiende Django AbstractUser)
tableros	Tableros Kanban con columnas configurables
tablero_usuarios	Membresía de usuarios en tableros + permisos granulares
tickets	Tickets/Notas dentro de un tablero
comentarios	Comentarios y trazabilidad de auditoría
notificaciones	Alertas para miembros del tablero

Requisitos previos

- [Docker](#) (20.10+)
- [Docker Compose](#) (v2+)

Despliegue con Docker Compose

1. Levantar los contenedores

```
docker compose up --build -d
```

Esto construye:

- El frontend (build de Vite + nginx)
- El backend (imagen Python 3.12 con Django)
- MariaDB 11.6 con volumen persistente

2. Verificar que todo está funcionando

```
# Ver estado de los contenedores
docker compose ps

# Ver logs del backend
docker compose logs -f backend

# Ver logs de MariaDB (esperar "ready for connections")
docker compose logs -f db
```

3. Acceder a la aplicación

Servicio	URL
Frontend	http://localhost
Backend API	http://localhost:8000/api/
Admin Django	http://localhost:8000/admin/
MariaDB	localhost:3306

4. Usuario administrador

El `entrypoint.sh` crea automáticamente el usuario admin:

Campo	Valor
Email	<code>admin@ticket.com.ar</code>
Contraseña	<code>EstoNoEsPass</code>

5. Credenciales de la base de datos

Campo	Valor
Usuario	<code>ticketera</code>
Contraseña	<code>ticketera</code>

Campo	Valor
Base de datos	ticketera
Puerto	3306

Desarrollo local (sin Docker)

Backend

```
cd backend
python -m venv venv
source venv/bin/activate
pip install -r requirements.txt

# Necesitás MariaDB/MySQL corriendo localmente
python manage.py migrate
python manage.py runserver
```

Frontend

```
npm install
npm run dev
```

El frontend en desarrollo (`npm run dev`) corre en `http://localhost:5173` y se conecta al backend en `http://localhost:8000/api`.

Comandos útiles

Comando	Descripción
<code>docker compose up --build</code>	Reconstruir y levantar todo
<code>docker compose down</code>	Detener los contenedores
<code>docker compose down -v</code>	Detener y eliminar volúmenes (borra la DB)
<code>docker compose logs -f backend</code>	Ver logs del backend en tiempo real
<code>docker compose exec backend python manage.py shell</code>	Shell de Django
<code>docker compose exec db mariadb -u ticketera -p</code>	Consola de MariaDB
<code>npm run dev</code>	Frontend en modo desarrollo
<code>npm run build</code>	Build de producción

API REST

El backend expone los siguientes endpoints bajo `/api/`:

Autenticación

Método	Endpoint	Descripción
POST	<code>/api/auth/login/</code>	Iniciar sesión
POST	<code>/api/auth/register/</code>	Crear cuenta
POST	<code>/api/auth/logout/</code>	Cerrar sesión
GET	<code>/api/auth/me/</code>	Obtener perfil del usuario actual
POST	<code>/api/auth/change-password/</code>	Cambiar contraseña
POST	<code>/api/auth/reset-password/</code>	Solicitar recuperación de contraseña

Recursos

Método	Endpoint	Descripción
GET/POST	<code>/api/usuarios/</code>	Listar/crear usuarios
GET/POST	<code>/api/tableros/</code>	Listar/crear tableros
GET/POST	<code>/api/tickets/</code>	Listar/crear tickets
GET/POST	<code>/api/comentarios/</code>	Listar/crear comentarios
GET/POST	<code>/api/notificaciones/</code>	Listar/crear notificaciones
GET/POST	<code>/api/tablero-usuarios/</code>	Gestionar membresías

Todos los endpoints de recursos soportan filtros por query parameters, por ejemplo:

- `GET /api/tickets/?tablero_id=1` — tickets de un tablero
- `GET /api/comentarios/?ticket_id=5` — comentarios de un ticket

Tecnologías

Frontend

- React 19.2
- Vite 7.3
- Tailwind CSS 4.2
- React Router DOM 7.13
- Recharts 3.9 (gráficos de estadísticas)
- @hello-pangea/dnd (drag & drop Kanban)

Backend

- Django 5.1
- Django REST Framework 3.15

- django-cors-headers 4.4
- mysqlclient 2.2 (driver MariaDB)
- Gunicorn 22.0

Infraestructura

- Docker / Docker Compose
- MariaDB 11.6
- nginx (servidor estático para el frontend)
- Python 3.12
- Node.js 20